

Name: M S GOWRISH SHANKAR REDDY

Roll No: 192424270

Date:25-8-26

Q1. Cartograms

Build a cartogram resized by the metric, alongside the true-area map showing average parcel delivery time (hours) across 7 states, for the logistics firm's Ops Head.

Code:

```
import json
import numpy as np
import matplotlib.pyplot as plt

with open("data.json") as f:
    d = json.load(f)

states = d["states"]
lon = np.array(d["lon"])
lat = np.array(d["lat"])
hours = np.array(d["hours"])

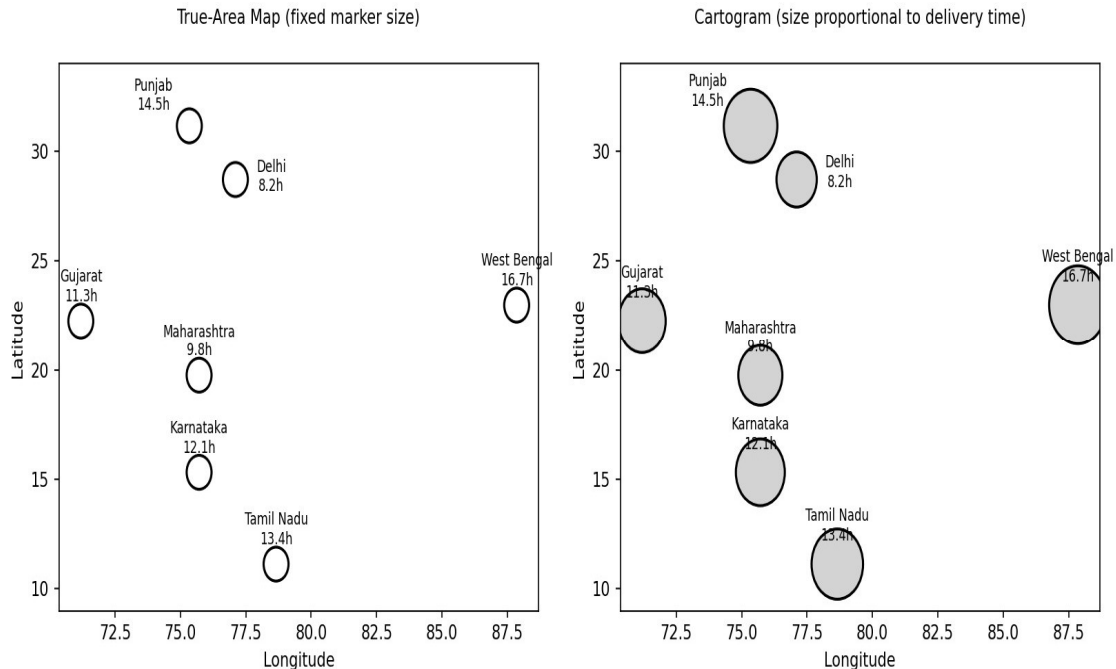
fig, axes = plt.subplots(1, 2, figsize=(11, 5))

ax1 = axes[0]
ax1.scatter(lon, lat, s=300, facecolors="none",
            edgecolors="black", linewidths=1.5)
for x, y, name, h in zip(lon, lat, states, hours):
    ax1.annotate(f"{name}\n{h}h", (x, y),
                textcoords="offset points", xytext=(0, 10),
                ha="center", fontsize=8)
ax1.set_title("True-Area Map (fixed marker size)")

ax2 = axes[1]
sizes = (hours / hours.max()) * 1600
ax2.scatter(lon, lat, s=sizes, facecolors="lightgray",
            edgecolors="black", linewidths=1.5)
for x, y, name, h in zip(lon, lat, states, hours):
    ax2.annotate(f"{name}\n{h}h", (x, y),
                textcoords="offset points", xytext=(0, 12),
                ha="center", fontsize=8)
ax2.set_title("Cartogram (size proportional to delivery time)")

plt.tight_layout()
plt.savefig("q1_cartogram.png", dpi=150)
```

Output:



Analysis:

In the true-area map, all seven states appear as equal-sized circles at their correct geographic position, so the eye has no cue to which state performs worse on delivery time until the printed labels are read one by one. In the cartogram, West Bengal (16.7h) and Punjab (14.5h) visibly swell into the largest circles while Delhi (8.2h) shrinks to the smallest, so the two slowest states are identified instantly without reading a single number. The cartogram trades true geographic shape and area for a directly readable ranking of delivery performance.

Recommendation for the Ops Head:

Use the cartogram view in weekly ops reviews to flag West Bengal and Punjab as the first states needing route or hub-capacity investigation, since their circle size alone signals the problem before any drill-down.

Q2. Interactive Geospatial Plots

Build an interactive map with hover, zoom, and filter controls showing average parcel delivery time (hours) across 7 states, for the logistics firm's Ops Head.

Code:

```
import json
import numpy as np
import matplotlib.pyplot as plt
```

```
with open("data.json") as f:
    d = json.load(f)
```

```
states = d["states"] lon =
np.array(d["lon"]) lat =
```

```

np.array(d["lat"]) hours =
np.array(d["hours"])

fig, axes = plt.subplots(1, 2, figsize=(12, 5.5))

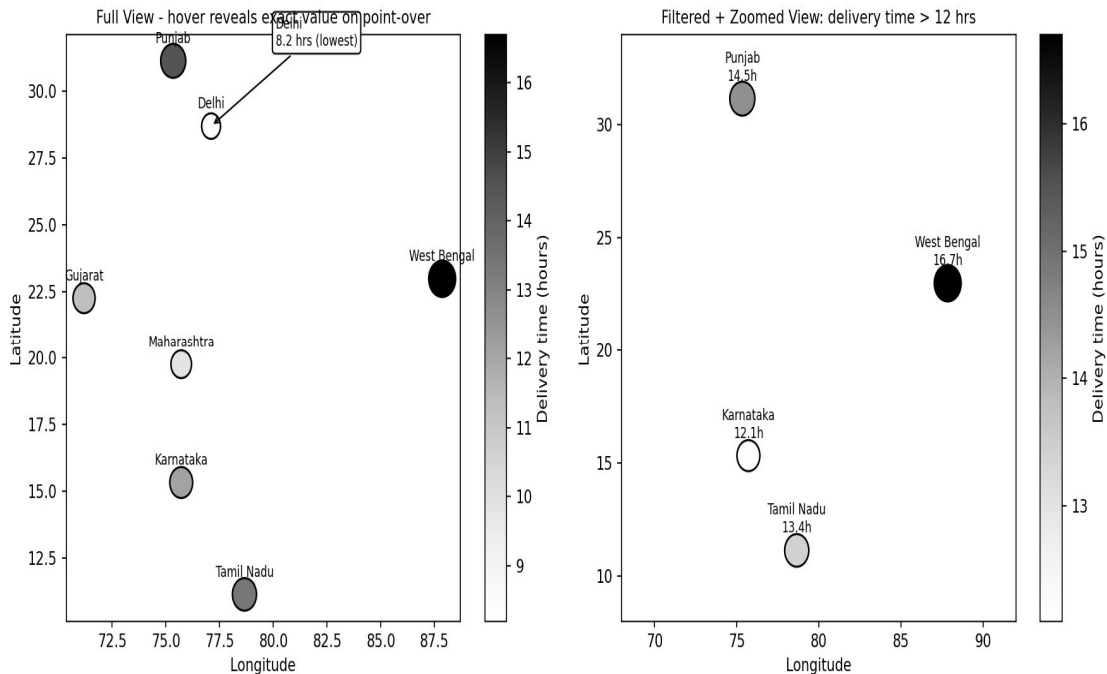
# Full interactive view (hover tooltip simulated on Delhi)
ax1 = axes[0] sizes = hours * 25 sc = ax1.scatter(lon, lat,
s=sizes, c=hours, cmap="Greys",
edgecolors="black", linewidths=1.2) for x, y, name in
zip(lon, lat, states):
    ax1.annotate(name, (x, y), textcoords="offset points",
xytext=(0, 10), ha="center", fontsize=8) i =
states.index("Delhi") ax1.annotate(f"Delhi\n{hours[i]} hrs
(lowest)", (lon[i], lat[i]), xytext=(lon[i] + 3, lat[i] + 3),
bbox=dict(boxstyle="round", fc="white", ec="black"),
arrowprops=dict(arrowstyle="->"), fontsize=8) plt.colorbar(sc,
ax=ax1)

# Filtered + zoomed view: delivery time > 12 hours ax2 =
axes[1] mask = hours > 12 sc2 = ax2.scatter(lon[mask],
lat[mask], s=hours[mask]*25, c=hours[mask],
cmap="Greys", edgecolors="black",
linewidths=1.2) for x, y, name, h in zip(lon[mask],
lat[mask], np.array(states)[mask],
hours[mask]): ax2.annotate(f"{name}\n{h}h", (x, y),
textcoords="offset points", xytext=(0, 10),
ha="center", fontsize=8) ax2.set_xlim(68, 92)
ax2.set_ylim(8, 34)
plt.colorbar(sc2, ax=ax2)

plt.tight_layout()
plt.savefig("q2_interactive.png", dpi=150)

```

Output:



Analysis:

The full view lets the operator hover over any state to read its exact delivery time on demand, for example pointing at Delhi reveals 8.2 hours, the fastest in the network, without cluttering the map with every number at once. Applying the filter control for delivery time above 12 hours removes Maharashtra, Delhi and Gujarat entirely and zooms into the remaining four states, immediately isolating Punjab, West Bengal, Karnataka and Tamil Nadu as the underperforming group. A static map cannot reproduce this: it would need four separate printed maps to show the same set of views.

Recommendation for the Ops Head:

Adopt the interactive dashboard for daily monitoring and set the filter's default threshold at 12 hours so the Ops Head opens the tool directly onto the states that require attention.

Q3. Choropleth Maps

Build a choropleth map of the regional metric showing average parcel delivery time (hours) across 7 states, for the logistics firm's Ops Head.

Code:

```
import json
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial import Voronoi
```

```
with open("data.json") as f:
    d = json.load(f)
```

```
states = d["states"]
lon = np.array(d["lon"])
lat =
```

```

np.array(d["lat"]) hours =
np.array(d["hours"])

points = np.column_stack([lon, lat])
bounding = np.array([
    [-1000, -1000], [-1000, 1000], [1000, -1000], [1000, 1000],
    [-1000, 25], [1000, 25], [78, -1000], [78, 1000]
]) all_points = np.vstack([points,
bounding]) vor = Voronoi(all_points)

fig, ax = plt.subplots(figsize=(7.5, 6.5)) norm =
plt.Normalize(hours.min(), hours.max()) cmap
= plt.cm.Greys

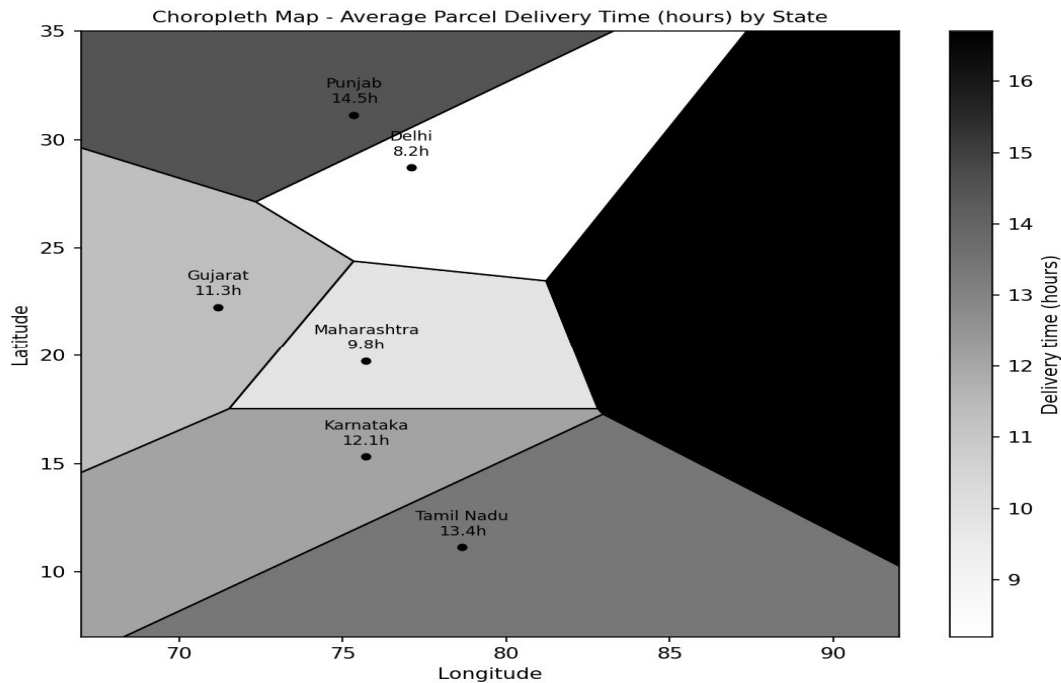
for i in range(len(points)):
    region = vor.regions[vor.point_region[i]]
    if -1 in region or len(region) == 0:
        continue    polygon = [vor.vertices[v] for v in region]
    ax.add_patch(plt.Polygon(polygon, facecolor=cmap(norm(hours[i])),
    edgecolor="black", linewidth=1))

ax.scatter(lon, lat, c="black", s=15) for x, y,
name, h in zip(lon, lat, states, hours):
    ax.annotate(f'{name}\n {h}', (x, y),          textcoords="offset
points", xytext=(0, 8),          ha="center", fontsize=8)
ax.set_xlim(67, 92) ax.set_ylim(7, 35)
plt.colorbar(plt.cm.ScalarMappable(cmap=cmap, norm=norm), ax=ax)
plt.tight_layout()

plt.savefig("q3_choropleth.png", dpi=150)

```

Output:



Analysis:

The eastern region occupied by West Bengal shades to the darkest tone on the map at 16.7 hours, standing out most clearly against the near-white Delhi region at 8.2 hours in the north. Since the metric is already an average (hours per parcel) rather than a raw count, the shading directly reflects service speed; had the map instead plotted a raw count such as total parcels delayed, a populous state like Maharashtra could appear artificially dark simply because it handles more volume, masking whether its per-parcel service is actually efficient.

Recommendation for the Ops Head:

Continue reporting delivery performance as a normalized average rather than a raw count, and prioritize a service-level audit for West Bengal, the darkest region on the choropleth.

Q4. 3D Geospatial Visualizations

Build a 3D extruded map with height encoding the metric showing average parcel delivery time (hours) across 7 states, for the logistics firm's Ops Head, and analyze 3D versus 2D choropleth for accurate comparison.

Code:

```
import json
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

with open("data.json") as f:
    d = json.load(f)

states = d["states"]
lon = np.array(d["lon"])
lat =
```

```

np.array(d["lat"]) hours =
np.array(d["hours"])

fig = plt.figure(figsize=(9, 7)) ax =
fig.add_subplot(111, projection="3d")

dx = dy = 0.6 norm =
plt.Normalize(hours.min(), hours.max()) colors
= plt.cm.Greys(norm(hours))

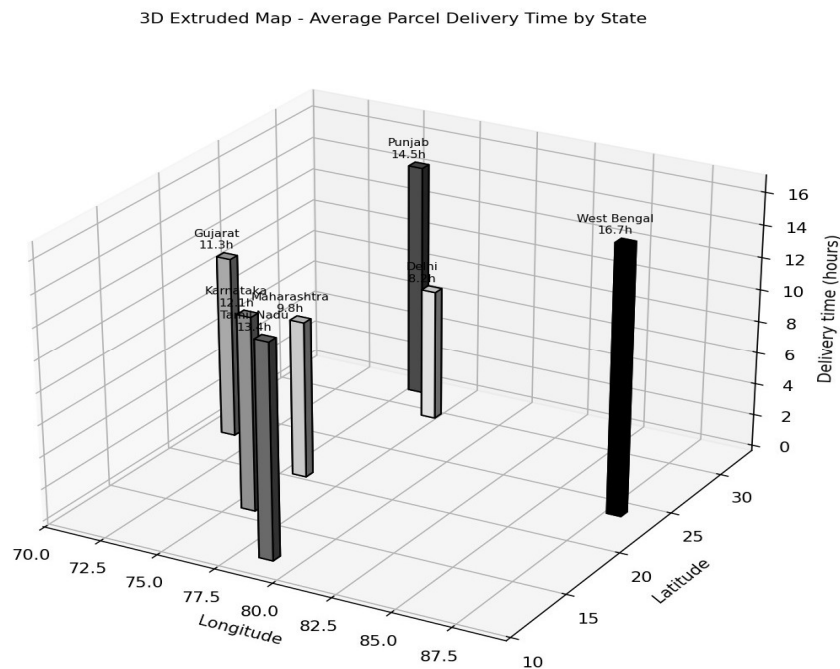
ax.bar3d(lon, lat, np.zeros(len(hours)), dx, dy, hours,
color=colors, edgecolor="black", shade=True)

for x, y, h, name in zip(lon, lat, hours, states):
    ax.text(x, y, h + 0.5, f'{name}\n{h}h', fontsize=7, ha="center")

ax.set_xlabel("Longitude")
ax.set_ylabel("Latitude")
ax.set_zlabel("Delivery time (hours)")
ax.view_init(elev=28, azim=-60)
plt.tight_layout()
plt.savefig("q4_3d.png", dpi=150)

```

Output:



Analysis:

The 3D extrusion makes the West Bengal bar visibly the tallest and the Delhi bar the shortest, so the overall best and worst performers are easy to spot. However, the closely clustered bars for Karnataka (12.1h), Tamil Nadu (13.4h) and Maharashtra (9.8h) partially occlude one another from this viewing angle, and perspective makes bars further from the camera look shorter than bars of the same height nearer to it. A 2D choropleth would encode the same values purely through flat, non-overlapping shading, so mid-range states could be compared without the risk of one bar hiding another or perspective distorting the reading.

Recommendation for the Ops Head:

Use the 3D view only for headline presentations that compare the clear extremes (West Bengal versus Delhi); rely on the 2D choropleth whenever the Ops Head needs to compare the middle-performing states accurately.

Q5. Layers

Build a multi-layer map combining a base layer, a choropleth layer, and point markers showing average parcel delivery time (hours) across 7 states, for the logistics firm's Ops Head.

Code:

```
import json
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial import Voronoi

with open("data.json") as f:
    d = json.load(f)

states = d["states"]
lon = np.array(d["lon"])
lat = np.array(d["lat"])
hours = np.array(d["hours"])

points = np.column_stack([lon, lat])
bounding = np.array([
    [-1000, -1000], [-1000, 1000], [1000, -1000], [1000, 1000],
    [-1000, 25], [1000, 25], [78, -1000], [78, 1000]
])
all_points = np.vstack([points, bounding])
vor = Voronoi(all_points)

fig, ax = plt.subplots(figsize=(7.5, 6.5))

# Layer 1: base layer
ax.add_patch(plt.Rectangle((66, 6), 27, 30, facecolor="0.92"))

# Layer 2: choropleth layer
norm = plt.Normalize(hours.min(), hours.max())
cmap = plt.cm.Greys
for i in range(len(points)):
    region = vor.regions[vor.point_region[i]]
    if -1 in region or len(region) == 0:
```



```

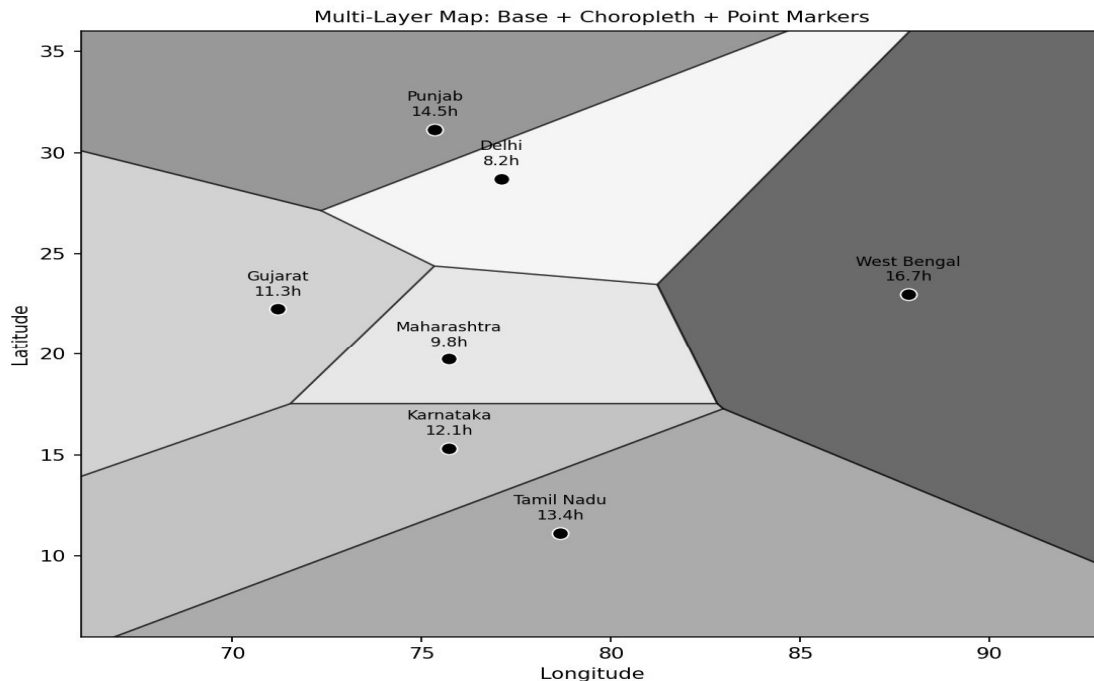
        continue
        polygon = [vor.vertices[v] for v in region]
ax.add_patch(plt.Polygon(polygon, facecolor=cmap(norm(hours[i])),
edgecolor="black", alpha=0.55))

# Layer 3: point markers
ax.scatter(lon, lat, c="black",
s=60, edgecolors="white") for x, y, name, h in zip(lon,
lat, states, hours):
    ax.annotate(f"{name}\n{h}h", (x, y),
        textcoords="offset points", xytext=(0, 9),
        ha="center", fontsize=8)

ax.set_xlim(66, 93)
ax.set_ylim(6, 36)
plt.tight_layout()
plt.savefig("q5_layers.png", dpi=150)

```

Output:



Analysis:

The base layer contributes only spatial context, a plain backdrop that anchors the seven states within the wider region. The choropleth layer adds the quantitative insight, letting the eye see at a glance that the east (West Bengal) and the north (Punjab) run slower than the west and south. The point-marker layer adds precision, pinning each state to its exact coordinate and exact hour value so the shaded region cannot be misread as covering the wrong state. None of the three layers is redundant here, but if the report were reduced to a single static print-out, the base layer would be the first candidate to drop since it adds no numeric information beyond what the choropleth boundaries already imply.

Recommendation for the Ops Head:

Keep all three layers in the live dashboard for full context, but when exporting a static one-page summary for the Ops Head, drop the plain base layer and keep only the choropleth and point markers to reduce clutter.

Q1 Cartograms — quick visual ranking of which states are lagging, useful for a fast weekly review rather than reading a table of numbers.

Q2 Interactive Plots — ability to drill into a specific state or filter to "problem states" (e.g., >12 hrs) without needing a new static report each time.

Q3 Choropleth — a single clear picture of regional performance to bring to a leadership meeting; also cares about the raw-count-vs-rate distinction since delivery time should be a fair per-parcel average, not skewed by volume.

Q4 3D — mostly a presentation/visual-impact tool for extremes (best vs worst state); they'd distrust it for fine-grained comparisons between similar-performing states.

Q5 Layers — wants the map decluttered for handoff to warehouse/hub managers, so knowing which layer is "nice to have" vs essential matters operationally.